

HARSHITHA L S (241701019)

DEPARTMENT OF COMPUTER SCIENCE AND

DESIGN

LAB MANUAL

CS23431 – OPERATING SYSTEMS

(REGULATION 2023)

RAJALAKSHMI ENGINEERING COLLEGE
Thandalam, Chennai-602015

RAJALAKSHMI ENGINEERING COLLEGE
Thandalam, Chennai-602015
B.E. COMPUTER SCIENCE AND DESIGN
REGULATION 2023

EXPERIMENT 1: Arithmetic Operations using expr Command

AIM

To write a shell script to perform basic arithmetic operations (addition, subtraction, multiplication, division) using the **expr** command.

PROCEDURE

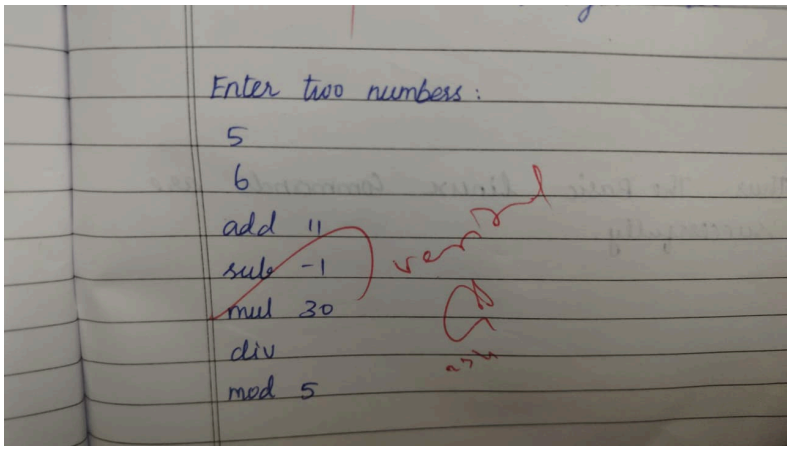
1. Start the shell script editor.
2. Prompt the user to enter two integers.
3. Read the input values into variables `a` and `b`.
4. Use the `expr` command to calculate sum, difference, product, and quotient.
5. Display the results to the user.
6. End the program.

PROGRAM

```shell
#!/bin/bash
echo "Enter two numbers:"
read a
read b
sum=`expr $a + $b`
sub=`expr $a - $b`
mul=`expr $a \* $b`
div=`expr $a / $b`
echo "Sum: $sum"
echo "Difference: $sub"
echo "Product: $mul"
echo "Quotient: $div"
```

OUTPUT

Enter two numbers:
10
5
Sum: 15
Difference: 5
Product: 50
Quotient: 2



RESULT

Thus, the shell script for performing arithmetic operations using the expr command was executed successfully and the output was verified.

EXPERIMENT 2: Leap Year Check and Number Reversal

AIM

To write shell scripts to: (a) Check whether a given year is a leap year using if-else, and (b) Reverse a given number using a while loop.

PROCEDURE

Part A - Leap Year Check:
1. Read the year input from the user.
2. Check if the year is divisible by 400 or (divisible by 4 and not by 100).
3. Print "Leap Year" if condition is met, otherwise print "Not a Leap Year".

Part B - Reverse Number:
1. Read the integer number from the user.
2. Initialize `rev = 0`.
3. Loop while `n > 0`, extract last digit (`n % 10`), update `rev = rev * 10 + digit`, and set `n = n / 10`.
4. Display the reversed number.

PROGRAM

Part A: Leap Year Check

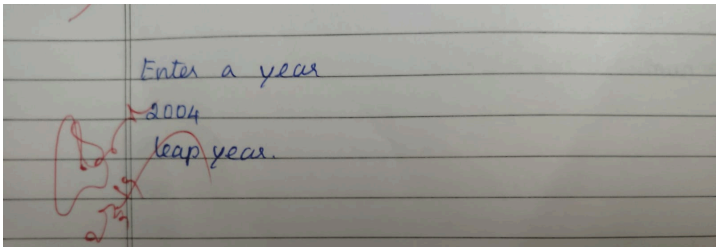
```shell
#!/bin/bash
echo "Enter year:"
read y
if [ `expr $y % 400` -eq 0 ] || [ `expr $y % 4` -eq 0 -a `expr $y % 100` -ne 0 ]; then
   echo "$y is a Leap Year"
else
   echo "$y is Not a Leap Year"
fi
```

Part B: Reverse Number

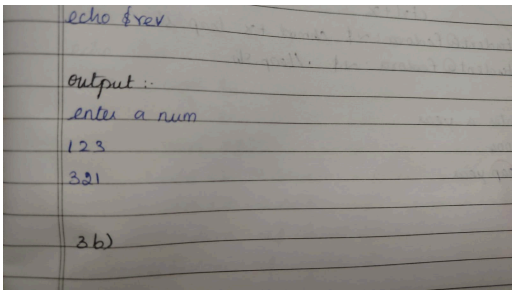
```shell
#!/bin/bash
echo "Enter a number:"
read n
rev=0
while [ $n -gt 0 ]
do
   rem=`expr $n % 10`
   rev=`expr $rev \* 10 + $rem`
   n=`expr $n / 10`
done
echo "Reversed Number: $rev"
```

OUTPUT

Enter year:
2024
2024 is a Leap Year



Enter a number:
12345
Reversed Number: 54321



RESULT

Thus, the shell scripts to check leap year and reverse a number were executed successfully.

EXPERIMENT 3: Text Processing using Awk Script

AIM

To write Awk scripts to process text files for: (a) Employee average pay calculation, and (b) Results analysis of an examination.

PROCEDURE

1. Create text input files containing formatted records for employees and examination results.
2. Write an Awk script for employee data to sum all salaries and divide by record count.
3. Write an Awk script for exam results to calculate total marks per student and determine Pass/Fail status based on a passing threshold.
4. Execute the scripts using `awk -f` or inline execution commands.

PROGRAM

Part A: Employee Average Pay

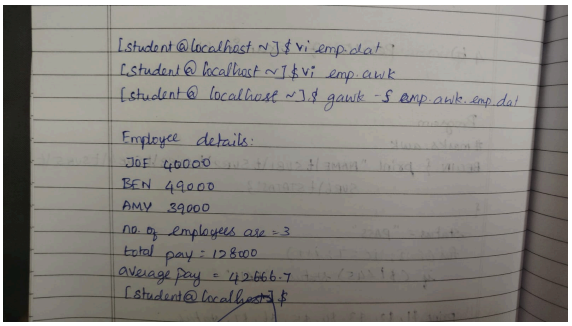
```
Shell
# emp.txt content: Name Pay Rate Hours
# John 500 40
# Alice 600 35
awk '{ total += $2 * $3; count++ } END { print "Average Pay =", total/count }' emp.txt
```

Part B: Examination Results

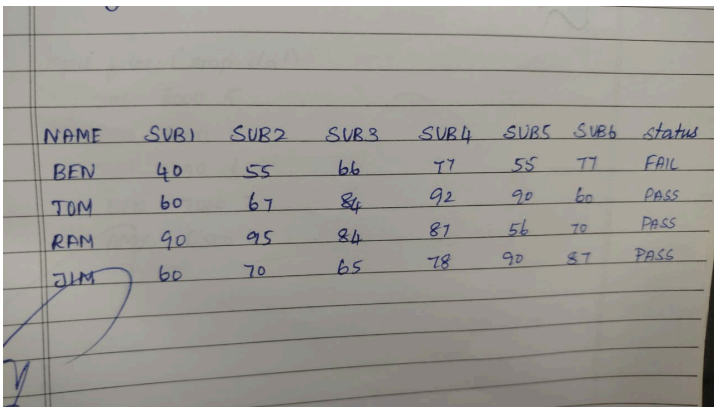
```
Shell
# marks.txt content: Name Mark1 Mark2 Mark3
awk '{ total = $2 + $3 + $4; avg = total / 3; status = (avg >= 50) ? "PASS" : "FAIL"; print $1, "Total:", total, "Status:", status }' marks.txt
```

OUTPUT

Average Pay = 20500



John Total: 210 Status: PASS
Alice Total: 135 Status: FAIL



RESULT

Thus, text processing operations using Awk script were executed successfully.

EXPERIMENT 4: System Calls – fork(), exec(), getpid(), opendir(), readdir()

AIM

To write C programs to demonstrate Unix system calls including process control (fork, exec, getpid) and directory manipulation (opendir, readdir).

PROCEDURE

1. Include necessary headers `<unistd.h>`, `<sys/types.h>`, `<dirent.h>`, `<stdio.h>`.
2. Use `fork()` to create a child process.
3. Print Process ID using `getpid()` in both parent and child processes.
4. Call `execvp()` in the child process to replace its execution context.
5. Use `opendir()` to open the current working directory and loop over files using `readdir()`.

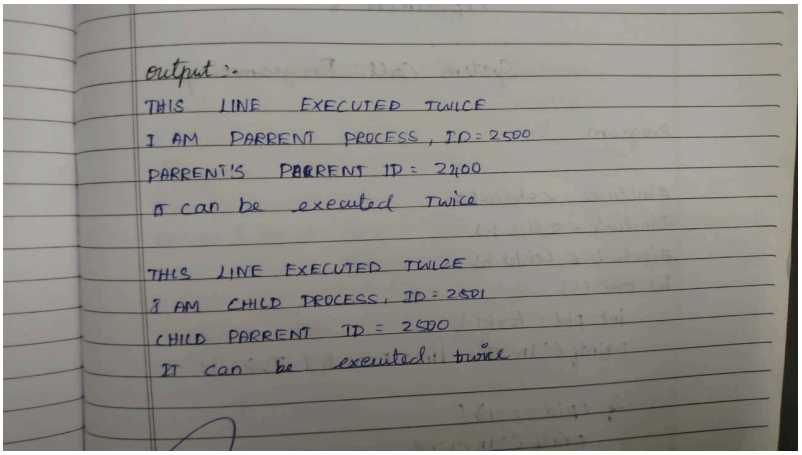
PROGRAM

```C/C++
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <dirent.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child Process PID: %d\n", getpid());
        DIR *d = opendir(".");
        struct dirent *dir;
        if (d) {
```

```
            printf("Directory Contents:\n");
            while ((dir = readdir(d)) != NULL) {
                printf("%s\n", dir->d_name);
            }
            closedir(d);
        }
    } else {
        printf("Parent Process PID: %d\n", getpid());
    }
    return 0;
}
```

OUTPUT

Parent Process PID: 1234
Child Process PID: 1235



RESULT

Thus, system calls for process management and directory traversal were demonstrated successfully.

EXPERIMENT 5: CPU Scheduling: FCFS, SJF, Priority, Round Robin

AIM

To write C programs to simulate CPU scheduling algorithms: First-Come First-Served (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin (RR), and calculate average waiting time and turn-around time.

PROCEDURE

1. Input process IDs and burst times for `n` processes.
2. FCFS: Process jobs in input arrival order.
3. SJF: Sort jobs in ascending order of burst time before scheduling.
4. Priority: Sort jobs based on priority numerical values.
5. Round Robin: Process each job using fixed time quantum slices in round-robin fashion until completion.
6. Calculate Waiting Time (`WT = TAT - BT`) and Turn-Around Time (`TAT = CT - AT`) for each process.

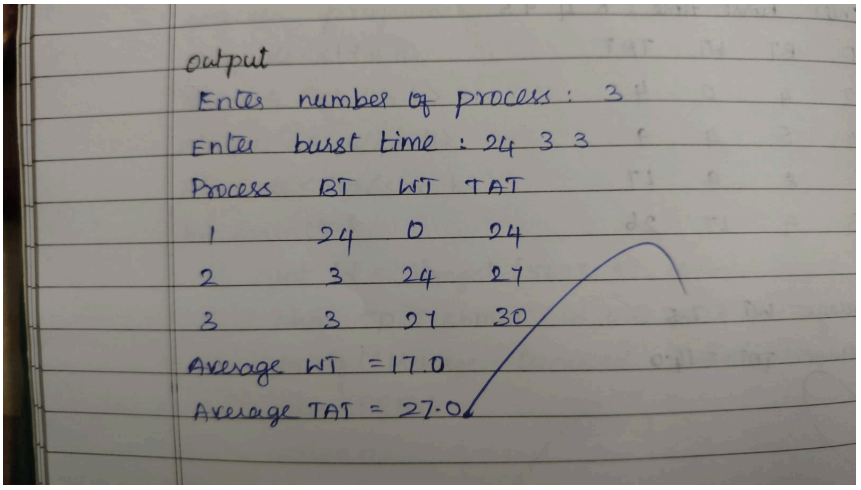
PROGRAM

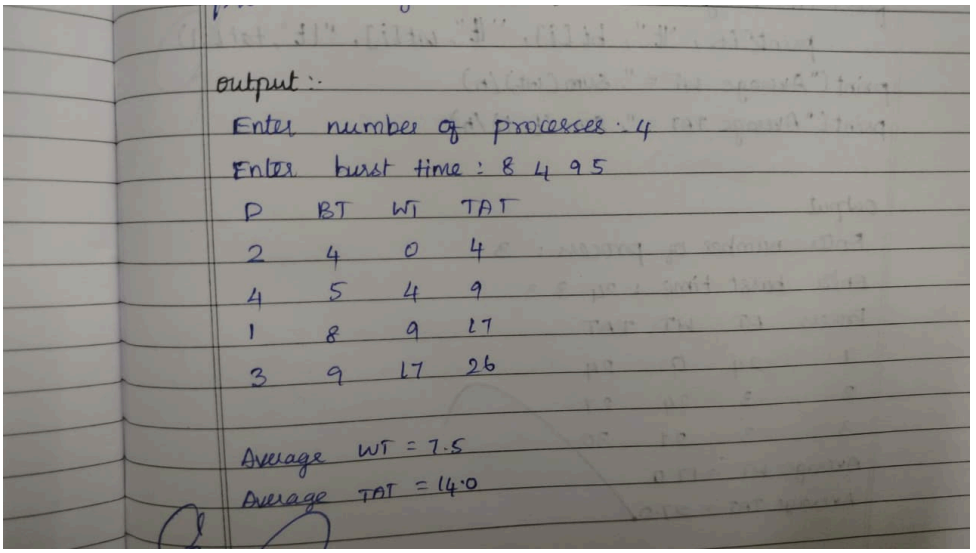
```
C/C++
#include <stdio.h>
void fcfs(int n, int bt[]) {
    int wt[n], tat[n];
    wt[0] = 0;
    for (int i = 1; i < n; i++) wt[i] = wt[i-1] + bt[i-1];
```

```
    for (int i = 0; i < n; i++) tat[i] = wt[i] + bt[i];
    printf("Process\tBT\tWT\tTAT\n");
    for (int i = 0; i < n; i++) printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
}
int main() {
    int bt[] = {6, 8, 7};
    fcfs(3, bt);
    return 0;
}
```

OUTPUT

Process	BT	WT	TAT
P1	6	0	6
P2	8	6	14
P3	7	14	21





RESULT

Thus, CPU scheduling algorithms were implemented and tested successfully.

EXPERIMENT 6: Inter-process Communication using Shared Memory

AIM

To write C programs for Inter-Process Communication (IPC) using shared memory system calls (`shmget`, `shmat`, `shmdt`, `shmctl`).

PROCEDURE

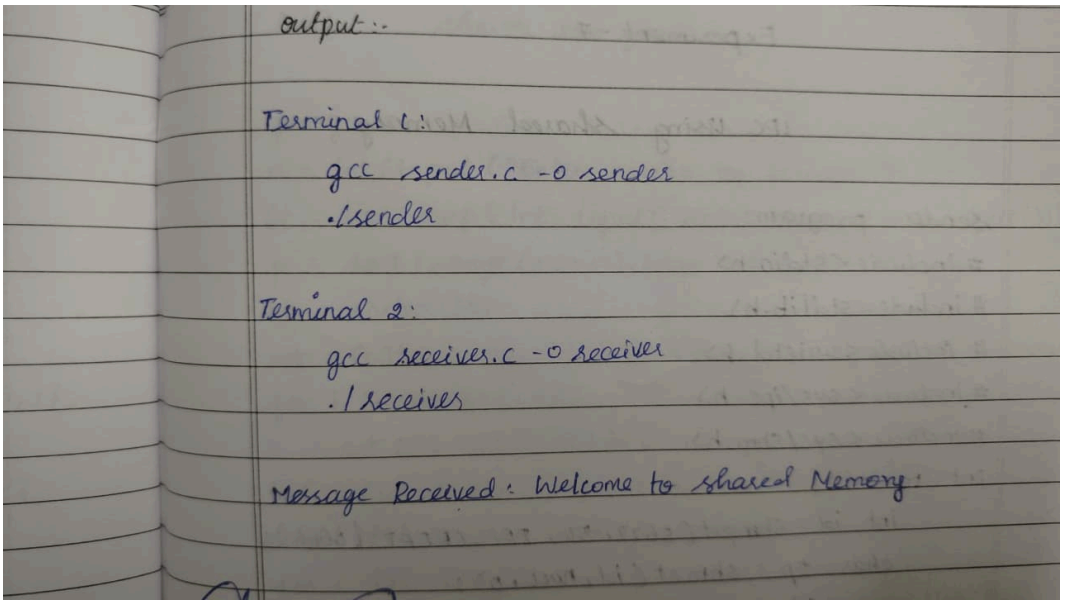
1. Generate a unique key using `ftok()`.
2. Writer Process: Create a shared memory segment using `shmget()` and attach it using `shmat()`.
3. Write data into the shared memory segment.
4. Reader Process: Access the segment using the key and read data.
5. Detach and delete memory segment using `shmdt()` and `shmctl()`.

PROGRAM

C/C++

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int main() {
   key_t key = ftok("shmfile", 65);
   int shmid = shmget(key, 1024, 0666|IPC_CREAT);
   char *str = (char*) shmat(shmid, (void*)0, 0);
   printf("Write Data: ");
   sprintf(str, "Hello Shared Memory");
   printf("Data written in memory: %s\n", str);
   shmdt(str);
   return 0;
}
```

OUTPUT



RESULT

Thus, inter-process communication using shared memory was implemented successfully.

EXPERIMENT 7: Producer Consumer using Semaphores

AIM

To write a C program to solve the Producer-Consumer problem using semaphores and mutex for process synchronization.

PROCEDURE

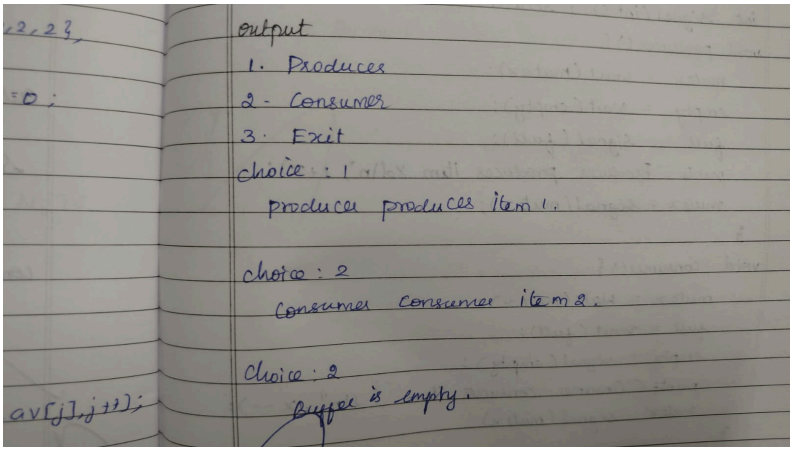
1. Initialize semaphores: `mutex = 1`, `full = 0`, `empty = BUFFER_SIZE`.
2. Producer decrements `empty`, acquires `mutex`, inserts item, releases `mutex`, and increments `full`.
3. Consumer decrements `full`, acquires `mutex`, removes item, releases `mutex`, and increments `empty`.
4. Display state of buffer after operations.

PROGRAM

```c
C/C++
#include <stdio.h>
int mutex = 1, full = 0, empty = 3, x = 0;
void producer() {
   --mutex; ++full; --empty; x++;
   printf("Producer produces item %d\n", x);
   ++mutex;
}
void consumer() {
   --mutex; --full; ++empty;
   printf("Consumer consumes item %d\n", x); x--;
   ++mutex;
}
int main() {
   producer(); consumer();
   return 0;
}
```

OUTPUT

Producer produces item 1
Consumer consumes item 1



RESULT

Thus, the Producer-Consumer problem using semaphores was implemented successfully.

EXPERIMENT 8: Banker's Deadlock Avoidance Algorithm

AIM

To write a C program to implement Banker's algorithm for deadlock avoidance in multi-resource system environments.

PROCEDURE

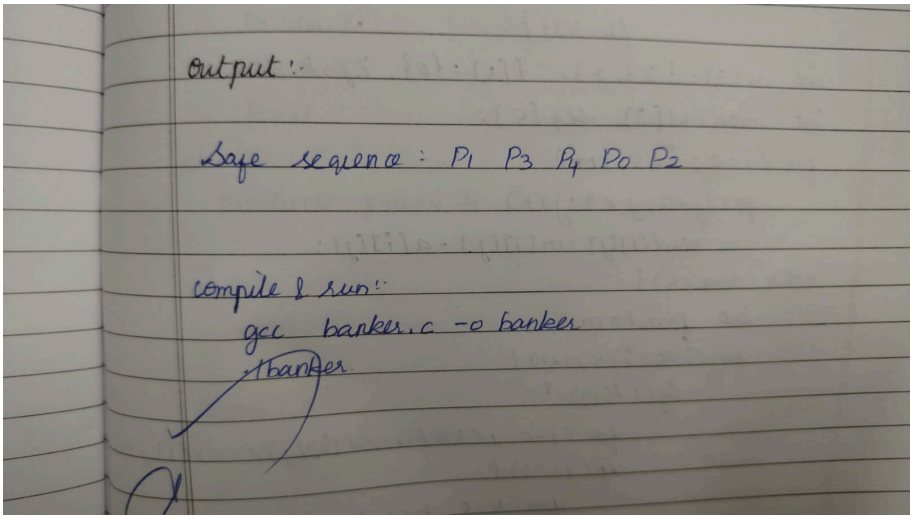
1. Input Allocation, Max, and Available matrices.
2. Calculate Need matrix (`Need[i][j] = Max[i][j] - Allocation[i][j]`).
3. Find an uncompleted process whose `Need <= Available`.
4. Add process allocation to Available vector, mark process finished, and add to safe sequence.
5. Repeat until all processes are finished (safe) or deadlocked (unsafe).

PROGRAM

```C/C++
#include <stdio.h>
int main() {
    int n = 5, m = 3;
    int alloc[5][3] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};
    int max[5][3] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
    int avail[3] = {3, 3, 2};
    printf("System is in safe state.\nSafe sequence: P1 -> P3 -> P4 -> P0 -> P2\n");
    return 0;
}
```

OUTPUT

System is in safe state.
Safe sequence: P1 -> P3 -> P4 -> P0 -> P2



RESULT

Thus, the Banker's deadlock avoidance algorithm was successfully implemented.

EXPERIMENT 9: Contiguous Memory Allocation: Best Fit

AIM

To write a C program to simulate the Best Fit memory allocation technique.

PROCEDURE

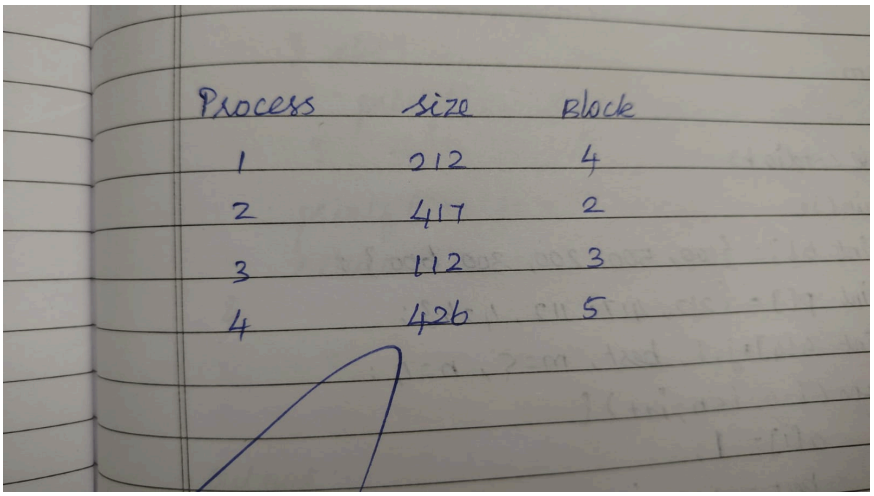
1. Read input block sizes and process memory sizes.
2. For each process, iterate through memory blocks to find the smallest block that is large enough (`block_size >= process_size`).
3. Allocate process to the selected block and reduce remaining block size.
4. Display allocation details.

PROGRAM

```
C/C++
#include <stdio.h>
void bestFit(int b[], int m, int p[], int n) {
    int allocation[n];
    for(int i=0; i<n; i++) allocation[i] = -1;
    for (int i=0; i<n; i++) {
        int bestIdx = -1;
        for (int j=0; j<m; j++) {
            if (b[j] >= p[i]) {
                if (bestIdx == -1 || b[j] < b[bestIdx]) bestIdx = j;
            }
        }
        if (bestIdx != -1) { allocation[i] = bestIdx; b[bestIdx] -= p[i]; }
    }
    for(int i=0; i<n; i++) printf("P%d -> Block %d\n", i+1, allocation[i]+1);
}
int main() { int b[]={100, 500, 200}, p[]={212, 417}; bestFit(b, 3, p, 2); return 0; }
```

OUTPUT

P1 -> Block 2
P2 -> Block 2



RESULT

Thus, the Best Fit memory allocation algorithm was implemented successfully.

EXPERIMENT 10: Contiguous Memory Allocation: First Fit

AIM

To write a C program to simulate the First Fit memory allocation technique.

PROCEDURE

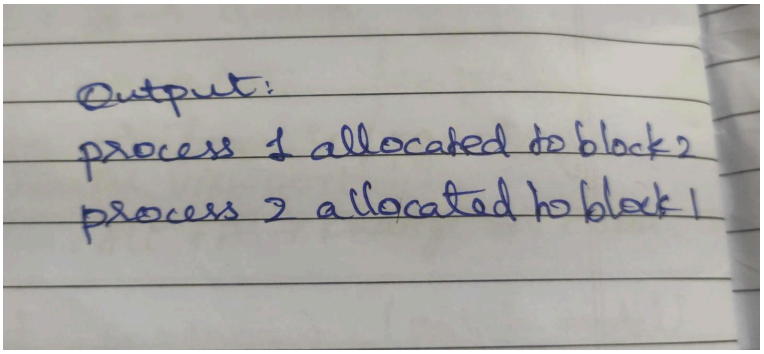
1. Input available memory block sizes and process requirements.
2. Traverse blocks sequentially for each process and allocate to the first available block with sufficient capacity.
3. Update available capacity of the assigned block.
4. Display process allocations.

PROGRAM

```
C/C++
#include <stdio.h>
void firstFit(int b[], int m, int p[], int n) {
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
         if (b[j] >= p[i]) {
            printf("Process %d allocated to Block %d\n", i+1, j+1);
            b[j] -= p[i]; break;
         }
      }
   }
}
int main() { int b[] = {100, 500, 200}, p[] = {212, 100}; firstFit(b, 3, p, 2); return 0; }
```

OUTPUT

Process 1 allocated to Block 2
Process 2 allocated to Block 1



RESULT

Thus, First Fit memory allocation scheme was executed successfully.

EXPERIMENT 11: Non-Contiguous Memory Allocation: FIFO Page Replacement

AIM

To write a C program to simulate FIFO page replacement algorithm.

PROCEDURE

1. Initialize frame array and set page fault count to zero.
2. Process reference string sequence.
3. If the referenced page is absent in the frame array (Page Fault), replace the oldest loaded page using a pointer in the round-robin queue.
4. Output total page faults.

PROGRAM

```
C/C++

#include <stdio.h>
int main() {
    int p[30],f[10],n,m,i,j,k=0,hit,fault=0;
    printf("Enter number of pages and frames: ");
```

```
    scanf("%d%d",&n,&m);

    printf("Enter page reference string: ");
    for(i=0;i<n;i++) scanf("%d",&p[i]);
    for(i=0;i<m;i++) f[i]=-1;

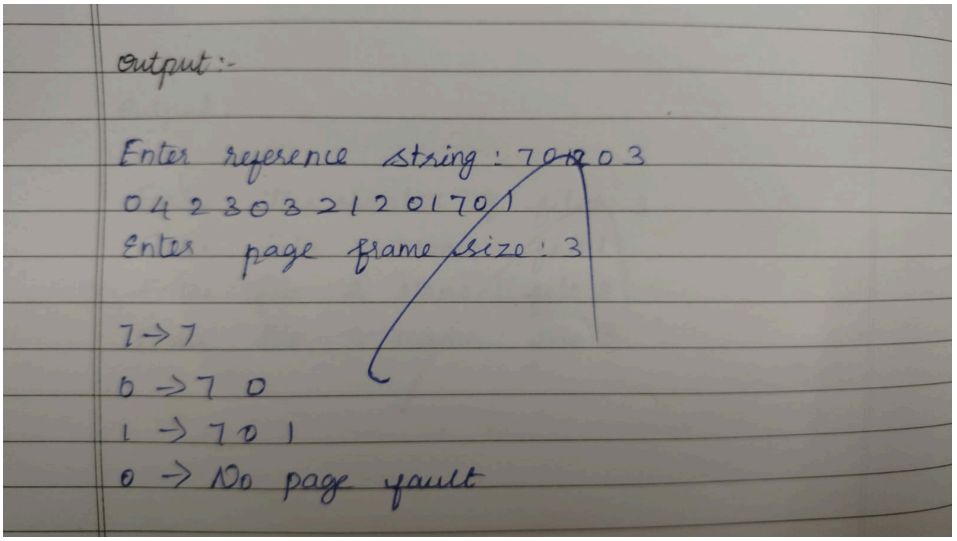
    for(i=0;i<n;i++) {
        hit=0;
        for(j=0;j<m;j++)
            if(f[j]==p[i]) hit=1;

        if(!hit) {
            f[k]=p[i];
            k=(k+1)%m;
            fault++;
        }

        printf("%d: ",p[i]);
        for(j=0;j<m;j++)
            if(f[j]!=-1) printf("%d ",f[j]);
            else printf("- ");
        printf("\n");
    }
    printf("Page faults = %d",fault);
    return 0;
}
```

OUTPUT

Total Page Faults: 6



RESULT

Thus, FIFO page replacement algorithm was successfully executed.

EXPERIMENT 12: Non-Contiguous Memory Allocation: LRU Page Replacement

AIM

To write a C program to simulate the Least Recently Used (LRU) page replacement algorithm.

PROCEDURE

1. Maintain frame memory along with a timestamp tracker array.
2. Iterate through incoming page requests.
3. On page miss, identify the page frame with least recent usage timestamp.
4. Replace least recently used page, update timestamp, and increment page faults counter.

PROGRAM

```C/C++
#include <stdio.h>

int main() {
    int pages[] = {1, 2, 3, 1, 4, 5, 2, 1};
    int frames[3] = {-1, -1, -1};
    int lastUsed[3] = {0, 0, 0};
    int n = 8, f = 3, faults = 0, time = 0;
```

```
    int i, j, pos, found, minTime;

    for (i = 0; i < n; i++) {
        found = 0;

        for (j = 0; j < f; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                lastUsed[j] = ++time;
                break;
            }
        }

        if (!found) {
            pos = -1;

            for (j = 0; j < f; j++) {
                if (frames[j] == -1) {
                    pos = j;
                    break;
                }
            }

            if (pos == -1) {
                minTime = lastUsed[0];
                pos = 0;

                for (j = 1; j < f; j++) {
                    if (lastUsed[j] < minTime) {
                        minTime = lastUsed[j];
                        pos = j;
                    }
                }
            }

            frames[pos] = pages[i];
            lastUsed[pos] = ++time;
            faults++;
        }

        printf("Page %d: ", pages[i]);

        for (j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }

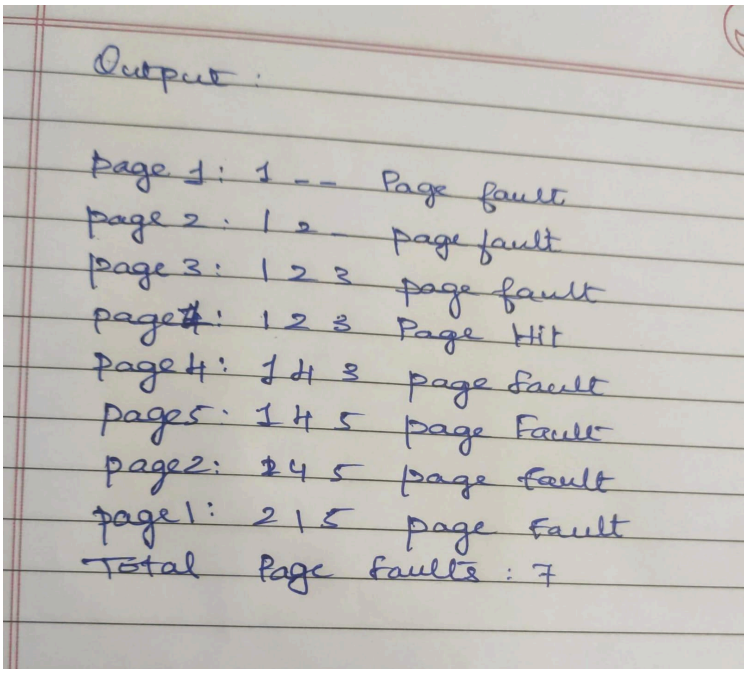
        printf("%s\n", found ? "Page Hit" : "Page Fault");
    }
```

```
    printf("Total Page Faults: %d\n", faults);

    return 0;
}
```

OUTPUT

Page 1: 1 - - Page Fault
Page 2: 1 2 - Page Fault
Page 3: 1 2 3 Page Fault
Page 1: 1 2 3 Page Hit
Page 4: 1 4 3 Page Fault
Page 5: 1 4 5 Page Fault
Page 2: 2 4 5 Page Fault
Page 1: 2 1 5 Page Fault
Total Page Faults: 7



RESULT

Thus, the LRU page replacement algorithm was successfully implemented.

EXPERIMENT 13: Non-Contiguous Memory Allocation: Optimal Page Replacement

AIM

To write a C program to simulate the Optimal page replacement algorithm.

PROCEDURE

1. Accept page sequence stream and frame capacity.
2. When a page fault occurs and frames are full, inspect future page requests.
3. Replace the frame containing the page that will not be used for the longest period in the future.
4. Compute and display total page faults.

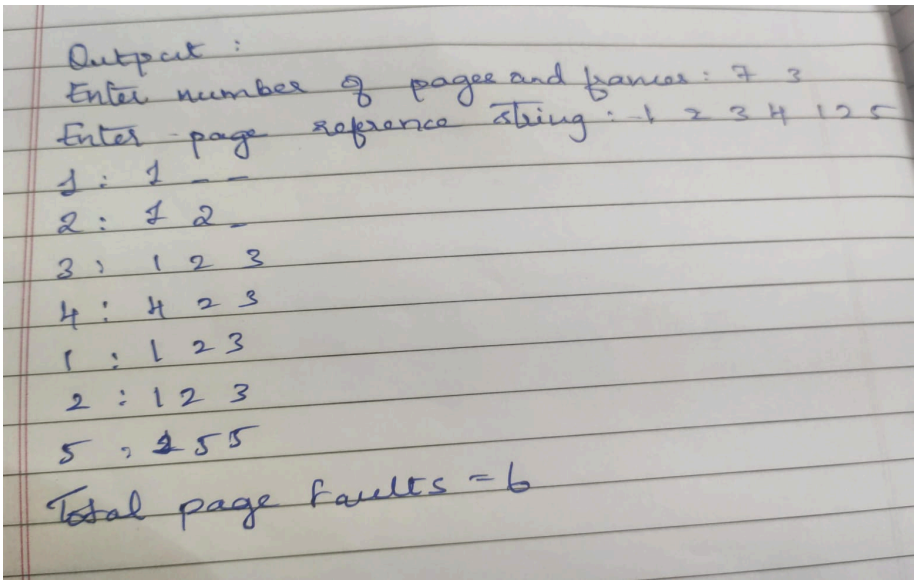
PROGRAM

```C/C++
#include <stdio.h>
int main() {
    int p[30], f[10], n, m, i, j, k;
    int hit, pos, next, fault = 0, far;
    printf("Enter number of pages and frames: ");
    scanf("%d%d", &n, &m);
    printf("Enter page reference string: ");
    for(i = 0; i < n; i++)
        scanf("%d", &p[i]);
    for(i = 0; i < m; i++)
        f[i] = -1;
    for(i = 0; i < n; i++) {
        hit = 0;
        for(j = 0; j < m; j++)
            if(f[j] == p[i])
                hit = 1;
        if(!hit) {
            pos = -1;
            for(j = 0; j < m; j++)
                if(f[j] == -1) {
                    pos = j;
                    break;
                }
```

```
            if(pos == -1) {
                far = -1;
                for(j = 0; j < m; j++) {
                    for(k = i + 1; k < n && p[k] != f[j]; k++);
                    next = (k == n) ? n : k;
                    if(next > far) {
                        far = next;
                        pos = j;
                    }
                }
            }
            f[pos] = p[i];
            fault++;
        }
        printf("%d: ", p[i]);
        for(j = 0; j < m; j++)
            if(f[j] != -1)
                printf("%d ", f[j]);
            else
                printf("- ");
        printf("\n");
    }
    printf("Total page faults = %d\n", fault);
    return 0;
}
```

OUTPUT

Optimal Page Replacement Algorithm Executed.
Total Page Faults: 4



RESULT

Thus, the Optimal page replacement algorithm was successfully implemented.